Software Architecture & Design

1. CORE CONCEPTS & DEFINITIONS

What is Architecture? (ගෘහ නිර්මාණ ශිල්පය යනු කුමක්ද?)
Architecture = **Structure + Organization + Plan**. එය පද්ධතියක කොටස් එකිනෙක සම්බන්ධ වන ආකාරය පෙන්වන **Blueprint** එකක් (සැලැස්මක්) ලෙස ක්‍රියා කරයි.

Software Architecture Definition:
සංකීර්ණ මෘදුකාංග පද්ධතියක high-level ව්‍යුහය, සංරචක (Components), ඒවා අතර අන්තර්ක්‍රියා (Interactions) සහ පද්ධතිමය සීමාවන් (Constraints) නිර්වචනය කිරීමයි. Focus එක වන්නේ *Big Picture* එක මිස එකිනෙක Code lines නොවේ.

```
Software Architecture Formula:
Architecture = Components + Interactions + Constraints
```

Real-World Analogy: Building vs Software Architecture

ගොඩනැගිල්ලක් තැනීමේදී මෙන්ම Software එකක් හදන විටද අදියර 3ක් ඇත:

1. **Requirements Gathering:** Client ගෙන් අවශ්‍යතා ලබා ගැනීම.
2. **Architecting (Design):** Blueprint / Structure එක නිර්මාණය කිරීම.
3. **Construction (Implementation):** සැලැස්මට අනුව coding / building කිරීම.

Note: Architect කෙනෙකු concrete හෝ glass නිෂ්පාදනය නොකරයි. ඔහු කරන්නේ පවතින ද්‍රව්‍ය (Materials) නිවැරදිව එකතු කර (Assemble) සේවාදායකයාගේ අවශ්‍යතාව ඉටුකිරීමයි.

When Do We Need Architecture?

සියලුම මෘදුකාංග සඳහා සංකීර්ණ Architecture එකක් අවශ්‍ය නොවේ. **Complexity (සංකීර්ණතාව)** මගින් Architecture එකක අවශ්‍යතාව තීරණය කරයි.

- **Small Apps / Scripts:** Simple structure standard එක ප්‍රමාණවත්ය.
- **Large / Scalable Systems:** Formal Software Architecture අනිවාර්ය වේ.

2. SOFTWARE ARCHITECTURE VS DESIGN

Attribute	Architecture	Design
Scope	Big Picture (High-level)	Detailed / Local logic
Focus	System Structure & Modules	Component Logic & Classes
Decisions	Long-term / Costly to change	Implementation level
Order	Comes First (අදියර 1)	Follows Architecture (අදියර 2)

3. ROLE OF A SOFTWARE ARCHITECT

Software Architect කෙනෙකුගේ ප්‍රධාන වගකීම් සහ කුසලතා:

- Overall system structure එක සැලසුම් කිරීම සහ High-level decisiones ගැනීම.
- ගැලපෙන Technologies, Frameworks සහ Databases තේරීම.
- Development team එකට මගපෙන්වීම සහ Quality Attributes (NFRs) තහවුරු කිරීම.

Key Skills Needed: Technical Knowledge, Problem Solving, System Thinking, Communication, Leadership & Management, Artistic Skill.

4. SYSTEM FLOW & COMPONENTS LOGIC

ඕනෑම නූතන Distributed System එකක් එකම program එකක් නොවේ. එය Components කිහිපයක එකතුවකි: **UI + Backend Logic + Database + Network**.

Data/Message Flow Architecture:

```
[User A] --> (Client App)
                |
            [Internet]
                |
         [Backend Server] <---> [Database]
                |                (Persistent Storage)
            [Internet]
                |
[User B] <-- (Client App)
```

1. **User A** app එක හරහා message එකක් යවයි.
2. App එක මගින් Internet හරහා **Backend Server** වෙත request එක යවයි.
3. Backend Server එක Business Logic සකසා **Database** එකේ Store කරයි.
4. Server එක Internet හරහා **User B** ගේ Client App එකට Data Push/Deliver කරයි.

5. REAL-WORLD CASE STUDIES

Case Study A: Sri Lankan Bus Booking System

- **Features:** Search buses, Book seats, Online payment.
- **Core Components:** User Mobile App, Booking Microservice, Payment Gateway Integration, Central Database.

Case Study B: Food Delivery System (e.g., UberEats)

- **Multiple Actors:** Customer, Restaurant, Delivery Rider.
- **Interactions:** Order processing, Real-time GPS tracking, Push notifications.

6. INTRODUCTION TO UML & VISUAL MODELING

පද්ධතියක් වචනයෙන් පමණක් පැහැදිලි කිරීම අමාරු නිසා **UML (Unified Modeling Language)** භාවිතා කරයි.

- **Use Case Diagrams:** System එක මගින් කරන්නේ කුමක්ද (What system does) සහ ඒ සමග සම්බන්ධ වන පාර්ශව (Who interacts - Actors) පෙන්වයි.
- **Component Thinking:** System එක Modules, Services, සහ Layers වලට කඩා වෙන්කර පාලනය කිරීම.

7. QUALITY ATTRIBUTES (NON-FUNCTIONAL REQUIREMENTS)

මෘදුකාංගයක් වැඩ කිරීම පමණක් ප්‍රමාණවත් නොවේ. Architecture මගින් පහත Quality Attributes තහවුරු කරයි:

- **Performance:** Response time සහ Processing speed.
- **Security:** Data protection සහ Authentication/Authorization.
- **Scalability:** Users ගණන වැඩිවන විට System එක handle කිරීමේ හැකියාව.
- **Maintainability:** System එක පහසුවෙන් update කිරීමේ සහ bugs හැදීමේ හැකියාව.

⚠️ **Common Student Pitfalls & Exam Traps:**

- **Trap 1:** Architecture සහ Design එකම දෙයක් ලෙස සැලකීම. (Exam tip: Architecture = High-level/Big Picture; Design = Low-level/Details).
- **Trap 2:** Architecture අවශ්‍ය වන්නේ සියලුම Programs වලට යැයි සිතීම. (Fact: Small/simple programs වලට heavy architecture අවශ්‍ය නොවේ).
- **Trap 3:** Software Architect යනු Code ලියන ප්‍රධාන Coder පමණක් ලෙස වැරදි ලෙස වටහා ගැනීම.

💡 **High-Yield Exam Tips & Revision Shortcuts:**

- **3 Constraints to Remember:** Time, Budget, Resources / Regulations.
- **Core Formula:** $Software\ System = Components + Interactions + Constraints$.
- **Quote to Remember for Essay Questions:** "Anyone can write code. Designing systems is what makes you an engineer."

📝 SELF-ASSESSMENT QUICK CHECK

1. **Q: Software Architecture සහ Design අතර ප්‍රධාන වෙනස කුමක්ද?**
   Ans: Architecture යනු System එකේ High-level structure එකයි (Big picture). Design යනු components වල internal details සහ implementation logic එකයි.
2. **Q: Software Architect කෙනෙකු සතුව තිබිය යුතු Non-technical skill එකක් සඳහන් කරන්න.**
   Ans: Communication, Leadership/Management, හෝ Artistic Skill.
3. **Q: Multi-user Message Application එකක Database එකේ ප්‍රධාන කාර්යය කුමක්ද?**
   Ans: Data (Messages, User details, Settings) දිර්ඝකාලීනව persistent ලෙස store කර තබා ගැනීම සහ retrieve කිරීම.
4. **Q: Software Quality Attributes 4ක් නම් කරන්න.**
   Ans: Performance, Security, Scalability, Maintainability.

> *"Anyone can write code. Designing systems is what makes you an engineer."*

Software Architecture & Design Principles

1. WHAT SOFTWARE ARCHITECTURE IS & ISN'T

Primitive Definition & Its Limitations

Primitive Definition: "*Software Architecture is a set of components and connections among them.*"

While a basic box-and-line diagram shows components, connections, and relationships, **it is NOT a complete software architecture** because it fails to specify crucial runtime properties.

```
Primitive Diagram Missing Information:
• Element Responsibilities: What does each component actually do?
• Connection Significance: Do lines mean invocation, data flow, control, synchronization, or secret hiding?
• Communication Mechanisms: Are APIs, HTTP, IPC, RPC, or Message Brokers used?
• Layout Significance: Does vertical layering imply containment, call hierarchy, or access permissions?
• Timing & Dynamic Behavior: Sequence of execution, latency, and throughput constraints.
```

Standard Academic Definition

According to *Bass, Clements, and Kazman (Software Architecture in Practice)*:

> **Formal Definition:** "The software architecture of a program or computing system is the **structure or structures** of the system, which comprise **software elements**, the **externally visible properties** of those elements, and the **relationships among them**."

Key Architectural Abstractions

- **Externally Visible Properties:** Assumptions other elements can make regarding an element (e.g., provided services, quality attributes, performance, error handling, shared resource usage). Internal implementation details are hidden.
- **Multiple Structures:** A system has multiple structures (e.g., functional decomposition for team assignment vs. runtime interaction patterns for deployment).
- **Universal Existence:** *Every computing system with software has an architecture*, even if it is undocumented, missing source code, or exists only as executable binaries.
- **Behavior:** Exposed via interfaces/methods; forms an essential part of the architectural specification.

2. ARCHITECTURAL ABSTRACTION LEVELS

Three distinct conceptual structures build up to a final software architecture:

```
Structural Mapping Derivation:
[ Reference Model ]   \
                       ==> [ Reference Architecture ] ==> [ Software Architecture ]
[ Architectural Pattern ] /
```

1. Architectural Pattern

A description of element types and relation types together with constraints on how they can be used. It is a *reusable template/schema*, not a complete system architecture.

- **Client-Server:** Client requests services; Server fulfills them using protocols like HTTP, SMTP, DNS.
- **Layered Architecture:** Organizes system into logical layers (Presentation, Business Logic, Data Access) for separation of concerns.
- **Microservices:** Decomposes system into small, independently deployable services communicating via APIs/brokers.
- **Event-Driven:** Decoupled components produce and consume asynchronous events (ideal for IoT/real-time systems).

2. Reference Model

A standard division of system functionality together with data flow between pieces. Solves a known domain problem by decomposing it into standard sub-problems.

- *Examples:* **OSI 7-Layer Model** (networking), **MVC** (UI software), **Von Neumann Architecture** (hardware computing model).

3. Von Neumann Architecture (Reference Model Example)

A hardware reference model consisting of 4 core subsystems connected via system buses:

- **Memory:** Stores both program instructions and data unified.
- **CPU (Control Unit + ALU):** Fetches/decodes instructions (CU) & executes arithmetic/logic (ALU).
- **Input/Output (I/O):** Interfaces with the external environment.

4. Reference Architecture

A Reference Model mapped directly onto software elements and data flows. It provides a blueprint for a specific domain.

3. CASE STUDY: FOOD DELIVERY SYSTEM

Applying architectural elements to a modern web application (e.g., Uber Eats):

Dimension	Architectural Implementation Details
Logical Structure	3 Layers: Frontend (Customer/Driver App), Business Logic (Order/Payment), Data Layer.
Physical Structure	Microservices in cloud containers connected via API Gateway & Kafka message broker.
Software Elements	User Service (Auth), Order Service, Delivery Assignment Service, Relational/NoSQL Databases.
Visible Properties	REST endpoints ($/api/v1/login$), latency guarantees, WebSocket real-time location pushes.
Relationships	Order Service publishes events to Kafka $\rightarrow$ Delivery Service assigns driver $\rightarrow$ Updates DB.

4. IMPORTANCE OF SOFTWARE ARCHITECTURE

Software architecture serves 3 primary functions:

1. **Vehicle for Stakeholder Communication:** Provides a high-level abstraction enabling mutual understanding, negotiation, and consensus among users, managers, architects, and developers.
2. **Earliest Set of Design Decisions:** Defines early choices that have the longest-lasting, most far-reaching effects, making them costly to reverse.
3. **Transferable Abstraction:** Allows reuse of architectural models across similar domains.

7 Far-Reaching Effects of Early Design Decisions

1. **Constraints Implementation:** Restricts tech stack and coding patterns (e.g., choosing Microservices prohibits monolith coding).
2. **Dictates Organizational Structure:** Direct application of **Conway's Law** (system structure mirrors organization structure).
3. **Enables Quality Attributes (NFRs):** Determines non-functional properties like scalability, security, availability (e.g., load balancers for uptime).
4. **Predicts System Qualities:** Allows architects to evaluate potential bottlenecks (e.g., single DB bottleneck) before coding.
5. **Eases Change Management:** Modularity isolates changes (e.g., updating payment API without touching Auth).
6. **Helps Evolutionary Prototyping:** Features can be incrementally added over a stable skeleton.
7. **Enables Accurate Estimation:** Simplifies cost and schedule planning based on component complexity.

> ⚠️ **COMMON EXAM PITFALLS & TRAPS**
> - **Trap 1:** Thinking "Architecture = Design Patterns". *Correction:* Architectural patterns operate at the system level; Design patterns operate at the code/class level.
> - **Trap 2:** Assuming undocumented systems have no architecture. *Correction:* EVERY system has an architecture, even if undocumented or binary-only!
> - **Trap 3:** Confusing Reference Models with Reference Architectures. *Formula:*
>
> $$\text{Ref Architecture} = \text{Ref Model} + \text{Software Elements}$$

> 💡 **HIGH-YIELD EXAM SHORTCUTS**
> - **Conway's Law:** Organizations design systems that mirror their own communication structure ($UI \rightarrow Backend \rightarrow DB teams$).
> - **Bass/Clements Definition Keywords:** Look for "*Elements*", "*Externally Visible Properties*", and "*Relationships*".
> - **Quality Attributes:** Architecture directly dictates NFRs (Performance, Security, Availability); code quality alone cannot fix bad architecture!

5. SELF-ASSESSMENT QUICK CHECK

Q1: What are "externally visible properties" of an architectural element?
Ans: Assumptions other elements can make about it (APIs, response time, resource usage) without knowing its internal implementation code.

Q2: Why is a primitive box-and-line diagram insufficient as an architectural specification?
Ans: It omits element responsibilities, communication protocols, runtime timing, dynamic interactions, and structural constraints.

Q3: State the mathematical relationship between Reference Model, Architectural Pattern, and Reference Architecture.
Ans: $Reference\ Model + Architectural\ Pattern \rightarrow{Mapped} Reference\ Architecture \rightarrow Software\ Architecture$.

Q4: Explain Conway's Law in the context of system architecture.
Ans: System structure mirrors the communication boundaries of the organization building it (e.g., 3 software layers lead to 3 distinct developer teams).

ARCHITECTURAL STRUCTURES & VIEWS

1. CORE CONCEPTS & DEFINITIONS

Modern software systems are too complex to comprehend in their entirety at once. To analyze and communicate architecture meaningfully, we must focus on specific perspective projections called **views**.

Analogy 1: Medical Perspective

A neurologist, orthopedist, cardiologist, and pediatrician examine different body systems (nervous, skeletal, circulatory, pediatric). Each view has distinct properties, but together they completely describe human body architecture.

Analogy 2: Building vs. Floor Plan

```
Structure = The real physical building (Classrooms, Labs, Library)
View      = The floor plan map shown to students (Representation)
```

Formal Definitions & Comparison

Concept	Definition	Key Focus
Structure	The set of software or hardware elements *as they exist* in reality.	Concrete realization (Source code, runtime objects, hardware).
View	A representation of a *logical subset* of elements and their relations.	Documentation tool for specific stakeholders.

Note: In practice, these two terms are often used interchangeably.

2. TAXONOMY OF ARCHITECTURAL STRUCTURES

Architectural structures are classified into **3 main categories** based on element types:

```
Architectural Structures
  ├── 1. Module Structures (Static / Design-Time)
  ├── 2. Component-and-Connector (C&C) Structures (Runtime)
  └── 3. Allocation Structures (Environment / Mapping)
```

3. CATEGORY I: MODULE STRUCTURES

Elements: Implementation units (modules) assigned based on functional responsibilities.

Questions Answered:

- Primary functional responsibility assigned to each module?
- What software elements is a module allowed to use / actually uses?
- How are modules related by generalization or specialization?

Sub-types of Module Structures

1. **Decomposition Structure:**
   - *Units:* Modules.
   - *Relation:* "is a submodule of" (recursive tree breakdown).
   - *Products:* Interface specs, source code, test plans.
   - *Decomposition Models:*
     a) **Object Model:** Interacting objects/classes (e.g., `Student`, `Lecturer`).
     b) **Pipeline / Data-flow Model:** Functional modules transforming inputs to outputs (e.g., `Login Module` → `Registration` → `Payment`).
2. **Uses Structure:**
   - *Units:* Modules, procedures, or interface resources.
   - *Relation:* "uses" (depends on correct operation of another).
   - *Purpose:* Building easily extensible systems.
3. **Layered Structure:**
   - *Units:* Layers (groups of related functionalities).
   - *Relation:* In strict layering, layer n may *only* use services of layer $n-1$.
   - *Example Tiering:*

```
Presentation Tier (ASP.NET Webforms)
      ↓ ↑
Business Tier (C# Classes & Calculation)
      ↓ ↑
Data Tier (SQL Server & Stored Procedures)
      ↓ ↑
SQL Server (Database)
```

4. **Class / Generalization Structure:**
   - *Units:* Classes.
   - *Relation:* "inherits-from" or "is-an-instance-of".

Use Cases: Modular development, maintainability, team task assignments.

4. CATEGORY II: C&C STRUCTURES

Elements: Executing runtime components and communication connectors.

Questions Answered: Major executing components? Shared data stores? Replicated elements? Data progression through system? Parallel execution pathways?

Sub-types of C&C Structures

- **Process Structure:** Dynamic aspect. Units are processes/threads connected via communication, synchronization, and/or exclusion operators. Relation is *attachment*. Guarantees execution performance & availability.
- **Concurrency Structure:** Connectors are "logical threads" mapped to physical threads. Helps identify parallelism and resource conflicts.
- **Shared Data Structure:** Components/connectors storing, creating, and accessing persistent data to maintain performance and data integrity.
- **Client-Server Structure:** Components are Clients and Servers; Connectors are network protocols and messages.
- **Other Views:** Pipe-and-Filter, Event-Driven, Peer-to-Peer.

Use Cases: Runtime behavior analysis, scalability, bottleneck identification.

5. CATEGORY III: ALLOCATION STRUCTURES

Focus: Relationship between software elements and external environments (hardware, files, teams).

Sub-types of Allocation Structures

1. **Deployment:** Maps software (processes) to hardware (processors) and communication pathways. Relations: `allocated-to`, `migrates-to`. Crucial for distributed system performance, security, and availability.
2. **Implementation:** Maps modules to source files/directory structures in version control/build systems.
3. **Work Assignment:** Maps module implementation responsibilities to development teams based on required expertise.

6. CASE STUDY: LIBRARY SYSTEM

Structure Group	Real-World Realization
Module	Login, Payment, Report modules.
C&C	Web Browser + App Server + DB interactions.
Allocation	App Server on AWS Cloud, DB on physical host.

⚠️ **Common Student Pitfalls & Exam Traps:**

- **Module vs. C&C Confusion:** Modules exist at compile/design-time (code units). C&C exists at runtime (processes, memory blocks).
- **Layered vs. Uses Trap:** Layered is a specific subset of "Uses". Layer n strictly uses $n-1$. "Uses" permits arbitrary dependencies.
- **Deployment vs. Implementation:** Deployment maps software to *hardware*; Implementation maps modules to *files/folders*.

💡 **High-Yield Exam Revision Shortcuts:**
Remember **"M-C-A"**: **M**odule = Code/Design organization | **C**&C = Runtime execution | **A**llocation = Environment mapping.

📝 SELF-ASSESSMENT QUICK CHECK

Q1: What is the main relation in a Decomposition Structure?

Ans: The "is a submodule of" relation.

Q2: Contrast the Harvard and Von-Neumann architectures in C&C terms.

Ans: Von-Neumann shares memory/bus causing structural hazards during simultaneous IF & MEM stages. Harvard uses separate instruction/data buses avoiding bottlenecks.

Q3: Which structure answers "What processor does each element run on?"

Ans: The **Deployment Structure** (an Allocation structure).

THE ARCHITECTURE BUSINESS CYCLE (ABC)

1. CORE THEORY & KEY DEFINITIONS

Software Architecture is shaped by a continuous feedback loop of technical, business, and social influences. In turn, the deployed architecture affects these very environments, driving future iterations. This bi-directional feedback mechanism is termed the **Architecture Business Cycle (ABC)** (Bass, Clements, & Kazman).

```
[Technical / Business / Social Factors]
              | (Influences)
              ▼
    [Software Architecture]
              | (Affects & Shapes)
              ▼
[Future Technical / Business / Business Environment]
```

The Three Primary Origins/Influences

- **Business Decisions:** Low development cost, employee retention, leveraging existing corporate assets, reducing time-to-market.
- **Technical Decisions:** Target hardware (e.g., touchscreens), rich functionality, modern GUIs, platform constraints.
- **Social/Organizational Decisions:** Team structures, expertise, and management policies.

2. STAKEHOLDER SPECTRUM & QUALITY ATTRIBUTES

Stakeholders are individuals or organizations interested in the construction of the system. Diverse stakeholders hold contrasting, often conflicting expectations. The **Architect** acts as a mediator balancing these needs.

Stakeholder Role	Primary Expectations & Goals
Management	Low cost, keep team employed, leverage existing corporate assets/ROI.
Marketing	Neat features, short time-to-market, low cost, parity with competitors.
End User	Optimal behavior, high performance, strong security, reliability, usability.
Maintenance Org	High modifiability, supportability, ease of future code changes.
Customer	Low purchase/operational cost, timely delivery, minimal breaking changes.

Key Quality Attribute Definitions

- **Performance:** Execution speed and task completion within time limits.
- **Reliability:** System's ability to operate without failure under defined conditions.
- **Availability:** System uptime and readiness for immediate user demand.
- **Modifiability:** The ease with which software changes can be executed.
- **Interoperability:** Ability to seamlessly exchange data with external systems.
- **Security & Usability:** Protection against invasion and ease of human operation.

3. ORGANIZATIONAL & ENVIRONMENTAL INFLUENCES

Influences by Developing Organization

- **Structure & Budget:** Available staff skills dictate architectural choices (e.g., idle client-server engineers leads to client-server design). Specialized subsystems may be subcontracted.
- **Immediate Business Influences:** Maximizing Return on Investment (ROI) by reusing established frameworks or internal microservices.
- **Long-Term Business Influences:** Strategic multi-project investments in foundational infrastructure (e.g., standardizing on Oracle DB or specific Cloud ERPs).

Influences by Architect's Experience & Tech Environment

- **Prior Experience & Biases:** Reusing familiar, successful patterns while shunning past failures. Reusing patterns saves time but can limit optimal novel choices.
- **Technical Environment:** Prevailing industry trends (e.g., Web-centric/microservices vs legacy monoliths) heavily constrain architectural selections.

4. REAL-WORLD CASE STUDY: NETFLIX ABC CYCLE

```
1. Initial State: Monolithic system struggling under scale.
2. Environmental Forces: Rapid global growth, demand for high uptime,
technical limits of monolith, market competition.
3. Architectural Decision: Migrated to Cloud-Native Microservices.
4. Business Impact: Rapid feature deployments, high fault tolerance,
team autonomy.
5. Cycle Continuation: Massive scale created new challenges (e.g.,
microservice orchestration), driving the NEXT architectural evolution
cycle.
```

5. SOFTWARE PROCESS & 7 CORE ACTIVITIES

Software Architecture bridges **Requirements Engineering** and **Detailed Software Design/Implementation** within the Software Lifecycle.

The 7 Core Architectural Activities

1. **Create Business Case:** Define profitability, target market, time-to-market, cost, and expected ROI.
2. **Understand Requirements:** Elicit functional/quality requirements via Use Cases, Finite-State Machines (for safety-critical systems), and Prototyping.
3. **Create/Select Architecture:** Choose patterns (Microservices, Client-Server, Layered, SOA).
4. **Communicate Architecture:** Unambiguously share models with Developers, Testers, and Management.
5. **Evaluate Architecture:** Compare candidate designs against Quality Attributes (Performance vs Scalability vs Maintainability trade-offs). Discard infeasible options early.
6. **Implement System:** Developers write code strictly following architectural constraints, protocols (e.g., REST), and patterns (e.g., Pub-Sub).
7. **Ensure Architectural Conformance:** Prevent non-conforming changes during maintenance.
   - **Architectural Drift:** Gradual divergence between implementation and architecture.
   - **Architectural Erosion:** Degraded quality due to improper, ad-hoc modifications.

6. ARCHITECTURAL DESIGN GUIDELINES

Note: No architecture is universally "good". E.g., Distributed Client-Server fits banking but fails in real-time Avionics flight control.

Process Recommendation Guidelines

- Lead with a single architect or a small, strongly led team.
- Follow structured design methods like **Attribute-Driven Design (ADD)**.
- Iterative & incremental design over Big Upfront Design (BUFD).
- Regular stakeholder reviews and prototyping to mitigate technical risks.

Structural Guidelines & Separation of Concerns (SoC)

- **Separation of Concerns (SoC):** Divide system into distinct, independent features via:
  1. *Modularity:* Break into discrete functional parts.
  2. *Encapsulation & Info Hiding:* Expose well-defined interfaces; hide implementation details.
- **Quality Tactics:** Apply specific architectural tactics (e.g., Redundancy for fault tolerance, Decoupling for maintainability).
- **Vendor Independence:** Avoid hard coupling to specific third-party tools/products.

⚠️ EXAM TRAPS & COMMON STUDENT PITFALLS

- **Trap 1: Drift vs. Erosion:** Architectural *Drift* refers to implementation diverging from design over time. Architectural *Erosion* is the physical degradation of quality attributes caused by ad-hoc patches.
- **Trap 2: "Universal Good Architecture":** Never claim a design pattern is universally superior. Always state that appropriateness depends entirely on business goals and quality constraints (e.g., Client-Server vs. Avionics).
- **Trap 3: Ignoring Non-Functional Attributes:** Architecture is primarily driven by Quality Attributes (NFRs like availability/security), NOT just functional use cases!

💡 HIGH-YIELD EXAM SHORTCUTS

- **ABC Definition Core:** Remember: `Environment` → `Architecture` → `System` → `Environment`.
- **Conflict Resolution:** When exam questions highlight conflicting stakeholder needs (e.g., Marketing's quick release vs Maintenance's clean code), state: *"The Architect must perform trade-off analysis and mediate."*
- **3 Elements of SoC:** Modularity + Encapsulation + Information Hiding.

📝 SELF-ASSESSMENT QUICK CHECK

1. **Q: How does immediate business influence differ from long-term business influence on architecture?**
   Ans: Immediate focuses on maximizing ROI by reusing existing code/frameworks. Long-term aligns with strategic foundational investments (e.g., enterprise Oracle DB or cloud standard) across future systems.
2. **Q: Differentiate between Architectural Drift and Architectural Erosion.**
   Ans: Drift occurs when the implementation diverges from the planned architecture. Erosion occurs when ad-hoc maintenance changes degrade the structural integrity and quality attributes of the system.
3. **Q: Why is a Client-Server pattern unsuitable for an Avionics flight control system?**
   Ans: Flight control demands deterministic, real-time response and low latency (Availability/Reliability), whereas Client-Server introduces network latency and distributed failure points.

Data Flow Architecture & Pipelines

1. CORE THEORY & SYSTEM OVERVIEW

In **Data-Flow Architecture**, an entire software system is viewed as a series of computational transformations performed on successive streams or batches of data. Execution is purely *data-driven*: the presence of data triggers and controls subsystem operations.

- **Decoupling & Independence:** Data sets and processing components are strictly decoupled. Subsystems do not share execution state and communicate solely via explicit data flow channels.
- **Architectural Topologies:** Represented as directed graphs—Linear (non-cyclic pipelines), Trees (branching/merging), or Graphs with Cycles (feedback structures).
- **Target Domain:** High-throughput computational processing with well-defined inputs and outputs, minimal real-time user interaction, and explicit data dependencies.
- **Key Quality Attributes:** Promotes maximum *Modifiability*, *Reusability*, and *Flexibility* across individual processing modules.

Three Primary Architectural Patterns

1. **Batch Sequential Systems:** Discrete, batch-oriented step-by-step execution.
2. **Pipe and Filter Architecture:** Stream-based, concurrent data transformation pipelines.
3. **Process Control Systems:** Feedback-driven architecture for real-time physical control.

2. BATCH SEQUENTIAL SYSTEMS

A classical pattern where execution moves in discrete, sequential steps. A module *cannot begin* its processing until the upstream module completes its computation on the **entire dataset**.

- **Data Transfer Mechanism:** Whole data batches are passed as intermediate files written to external storage (hard disk/file system).
- **Control & Execution Flow:** Externally driven; independent executable programs run to completion sequentially without concurrent overlap.

```
Batch Pipeline Architecture:
[Input File] ➔ [Validate Step] ➔ (Valid.txt) ➔ [Sort Step] ➔ (Sorted.txt) ➔ [Report Generator] ➔ [Final File]
```

Typical Real-World Applications

- Banking end-of-day batch transaction processing.
- Utility billing systems (monthly electricity/water meter processing).
- Traditional compiler stages (Batch preprocessing ➔ Syntax check ➔ Machine code gen).
- ETL (Extract, Transform, Load) pipelines in data warehousing.

Pros & Cons (Exam Focus)

Benefits (+)	Limitations (-)
• Simple division of tasks • High component reusability • Standalone executable programs • Low primary RAM requirements	• High I/O latency (disk writes/reads) • **Zero concurrency** (low throughput) • No interactive feedback • High latency for initial output

3. PIPE & FILTER ARCHITECTURE

Decomposes a system task into multiple independent functional components called **Filters** connected by directional communication channels called **Pipes**.

Core Architectural Components

- **Data Source (Pump):** Generates data streams (e.g., Sensor stream, DB query result, file feed).
- **Filter:** Independent processing element that transforms, enriches, or filters data incrementally. Filters know only their immediate input/output pipe interfaces.
- **Pipe:** Directional data channel (connector) backed by a FIFO buffer that queues data streams between filters.
- **Data Sink:** Final consumer/collector of processed data streams (e.g., UI display, database, log file).

```
Pipe & Filter Visual Topology:
(Source/Pump) =Pipe=> [ Filter 1 ] =Pipe=> [ Filter 2 ] =Pipe=> (Data Sink)
```

Filter Execution Modes & Stream Mechanics

- **Incremental Processing:** Filters process data chunk-by-chunk (tokens/stream items) immediately upon arrival without waiting for the full dataset.
- **Active Filter:** Runs in its own execution thread/loop; actively pulls data from input pipes and pushes output data down output pipes.
- **Passive Filter:** Driven passively by external callers; relies on upstream components to push data or downstream components to pull data.
- **Data Dynamics:** Push-Only (Producer-driven), Pull-Only (Consumer-driven), or hybrid Push/Pull pipelines.

```
Unix Shell Pipeline Example:
cat server.log | grep "500 ERROR" | awk '{print $4}' | sort | uniq -c
```

Real-World Applications

Unix/Linux command-line pipelines, media streaming codecs (audio/video DSP filters), stream analytics engines (Apache Flink, Kafka Streams), Splunk log analysis, compiler internal pipelines.

Pros & Cons Comparison Matrix

Benefits (+)	Limitations (-)
• **High Concurrency:** Filters execute in parallel • High Throughput & Low Latency • Hot-swappable filter components • Dynamic topology reconfiguration	• Not suited for interactive GUIs • Data parsing/marshalling overhead • Filter ordering dependencies • Error handling complexity

> 💡 **High-Yield Architecture Comparison:** Batch Sequential passes *entire files on disk* after full completion; Pipe & Filter streams *incremental tokens through FIFO memory queues* concurrently!

Process Control, Math Models & Implementations

4. PROCESS CONTROL SYSTEMS

Used to maintain a software/hardware system in a continuous desired state by constantly monitoring environmental variables and applying real-time closed-loop adjustments.

Key Variable Definitions & Taxonomy

- **Process:** The system/environment under control (e.g., Chemical reactor, HVAC room, Cruise control).
- **Controlled Variable (CV):** The physical state/output that the system aims to maintain (e.g., Room Temperature, Vehicle Speed).
- **Set Point (SP):** The target or desired numerical value for the CV (e.g., 22°C, 100 km/h).
- **Input Variables (Disturbances):** External environmental influences that cause fluctuations in CV (e.g., Sun exposure, outside temp, road incline).
- **Manipulated Variable (MV):** The variable modified directly by the controller to influence the CV (e.g., Heater power output, fuel valve flow rate).
- **Sensors & Actuators:** Sensors measure $CV(t)$; Actuators physically adjust $MV(t)$ under controller commands.

5. KEY FORMULAS & LATENCY LOGIC

1. Closed-Loop Feedback Error Calculation

The system feedback error $e(t)$ at any time step t is calculated as:

$$e(t) = SP - PV(t)$$

Where SP is the Set Point and $PV(t)$ is the Measured Process Value (CV).

2. Proportional-Integral-Derivative (PID) Formula

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau)d\tau + K_d \frac{de(t)}{dt}$$

Where $u(t)$ is the **Manipulated Variable Output**, K_p is Proportional gain, K_i is Integral gain, and K_d is Derivative gain.

3. Latency & Throughput Comparison Formulas

- **Batch Sequential Total Latency (T_{batch}):**

$$T_{\text{batch}} = \sum_{i=1}^{N} t_{\text{proc}}(i) + \sum_{i=1}^{N-1} t_{\text{disk\_IO}}(i)$$

- **Pipe & Filter Bottleneck Cycle Time (T_{pipe}):**

$$\text{Pipelined Cycle Time} = \max(t_1, t_2, \ldots, t_n)$$

6. THREADED PIPE & FILTER LOGIC (PYTHON)

```
import threading, queue

pipe = queue.Queue() # FIFO Buffer Pipe

class ActiveFilter1(threading.Thread):
    def run(self):
        for data in [1, 2, 3, 4, 5, 6]:
            if data % 2 != 0: # Filter logic
                pipe.put(data) # Push token
        pipe.put(None) # EOF Sentinel Signal

class ActiveFilter2(threading.Thread):
    def run(self):
        acc = 0
        while True:
            val = pipe.get()
            if val is None: break # Stop on EOF
            acc += val
            print(f"Val: {val}, Sum: {acc}")

f1, f2 = ActiveFilter1(), ActiveFilter2()
f1.start(); f2.start()
f1.join(); f2.join()
```

7. STEP-BY-STEP EXECUTION TRACE

Scenario: Trace processing of dataset `[10, 15, 20, 25, 30]` through an `OddFilter` and `AccumulatorFilter` pipeline.

Step	Input Data	OddFilter Output	Queue State	Accumulator Sum
1	10	Filtered Out	[]	0
2	15	15 (Passed)	[15]	$0 + 15 = 15$
3	20	Filtered Out	[]	15
4	25	25 (Passed)	[25]	$15 + 25 = 40$
5	30	Filtered Out	[]	40

⚠️ **Common Exam Pitfalls & Misconceptions:**

- **Batch vs Pipe & Filter:** Batch Sequential requires entire files written to disk before the next step starts. Pipe & Filter streams tokens continuously in RAM.
- **Active vs Passive Filters:** Active filters run on independent threads and control their own I/O loops. Passive filters are invoked passively via procedure calls.
- **Disturbance vs Manipulated Variable:** Ambient temperature is an *Input Variable (Disturbance)*. Heating coil electrical power is the *Manipulated Variable*!

💡 **High-Yield Architecture Summary Matrix:**

Architecture	Unit of Data	Execution	Latency
Batch Sequential	Whole File Batch	Sequential	Very High
Pipe & Filter	Stream Token	Concurrent	Low
Process Control	Continuous State	Closed Loop	Real-time

8. 📝 SELF-ASSESSMENT PRACTICE QUESTIONS

1. **Q: Why does Pipe & Filter achieve higher throughput than Batch Sequential?**
   Ans: Pipe & Filter allows filters to operate concurrently on streaming data tokens in RAM, eliminating disk I/O wait times.
2. **Q: In an HVAC controller with $SP = 24^\circ\text{C}$ and $PV = 19^\circ\text{C}$, calculate the Error $e(t)$.**
   Ans: $e(t) = SP - PV = 24 - 19 = +5^\circ\text{C}$.
3. **Q: What is the purpose of a Sentinel value in a FIFO pipe buffer?**
   Ans: Signals End-Of-File (EOF) to downstream filters so active reading threads can terminate gracefully.

DATA FLOW DIAGRAMS (DFD)

Structured Systems Analysis & Design Method (SSADM) | High-Yield Exam Guide

ICT 1022 | Lecture 06
Dr. WMKS Ilmini | Dept. of Computing

1. CORE THEORY & KEY CONCEPTS

A **Data Flow Diagram (DFD)** is a fundamental graphical modeling tool used in **SSADM** to represent the flow of data through an information system. It models system inputs, processes, data storage, and outputs without specifying underlying hardware or execution timing.

Primary Objectives of DFDs

- Graphically boundary system scope and external interfaces.
- Decompose complex systems into modular, structured processes.
- Show data sources, destinations, and data transformation steps.
- Serve as a communication bridge between analysts and users.

2. THE FOUR FUNDAMENTAL COMPONENTS

1. **External Entities (Source / Sink):** Objects outside the system boundary that interact with the system by providing (Source) or receiving (Sink) data.
   • *Naming standard:* Singular Noun (e.g., `Customer`, `Bank`).
2. **Processes:** Operations that transform incoming data into outgoing data.
   • *Naming standard:* Active Verb + Noun phrase (e.g., `Generate Bill`).
3. **Data Stores:** Repositories where data is held temporarily or permanently for later processing.
   • *Naming standard:* Plural Noun phrase (e.g., `Customer Records`).
4. **Data Flows:** Pipelines carrying data packets between components.
   • *Naming standard:* Noun phrase describing data (e.g., `Order Details`).

3. METHODOLOGICAL NOTATIONS COMPARISON

Different methodology standards use distinct shapes for DFD components:

Component	Yourdon & DeMarco	Gane & Sarson	SSADM Standard
Entity	Rectangle	Shaded Rectangle	Oval / Rectangle
Process	Circle / Bubble	Rounded Rect (Split)	Rectangle (ID Top)
Data Store	Parallel Lines	Open Rect (Left Box)	Open Rectangle
Data Flow	Directed Arrow	Directed Arrow	Directed Arrow

Data Store Identification Prefixes

In SSADM, data store IDs feature prefix codes indicating storage media:

- **D:** Computerized / Digital Store (e.g., `D1/Customer DB`)
- **M:** Manual File / Physical Cabinet (e.g., `M2/Ledger Book`)
- **T:** Transient / Temporary Buffer (e.g., `T1/Session Cache`)

4. FUNDAMENTAL CONNECTION RULES

Strict structural constraints govern valid connections in DFDs:

- **Mandatory Process Node:** Data cannot move or change without passing through a process.
- **Input/Output Rule:** Every process MUST have at least 1 input AND at least 1 output flow.
- **Data Conservation:** Output data must be derivable from input data.

5. DFD CONNECTION VALIDATION MATRIX

Connection Path	Validity	Rule Explanation
Process → Process	**VALID**	Direct flow of transformed data
Process → External Entity	**VALID**	System output sent to external sink
External Entity → Process	**VALID**	External input entering the system
Process → Data Store	**VALID**	Database write / update operation
Data Store → Process	**VALID**	Database read / query operation
Entity → Entity	**INVALID**	Outside system scope (no process)
Entity → Data Store	**INVALID**	Requires a process to write data
Data Store → Data Store	**INVALID**	Requires a process to move data

6. FUNCTIONAL DECOMPOSITION & LEVELS

DFDs use top-down functional decomposition to break system complexity down into manageable detail levels:

```
Level 0: Context Diagram
   |  (Process 0: Entire System, External Entities,
High-level Flows)
  ▼
Level 1: System DFD
   |  (Processes 1.0, 2.0, 3.0... + Data Stores
Introduced)
  ▼
Level 2: Sub-Process Decomposition
   |  (Sub-processes: 1.1, 1.2, 1.3... for Process 1.0)
  ▼
Level 3: Primitive DFD
     (Detailed Activities: 1.1.1, 1.1.2... Elementary
Processes)
```

Detailed Level Specifications

- **Context Diagram (Level 0):** Defines the system boundary. Shows the system as a single process node (Process ID `0`). Shows external entities and major inputs/outputs. **Data stores are NEVER shown at Level 0!**
- **Level 1 DFD:** Explodes Process `0` into major sub-systems (***1.0, 2.0, 3.0***). Introduces all primary Data Stores (***D1, D2***). Retains all Level 0 external entities.
- **Level 2 DFD:** Explodes a single Level 1 process (e.g., Process ***2.0***) into detailed steps (***2.1, 2.2, 2.3***). External entities are optional if internal.
- **Level 3 DFD (Primitive):** Decomposes Level 2 processes (***2.1***) into elementary functional activities (***2.1.1, 2.1.2***) which can be documented via Structured English.

7. THE LEVEL BALANCING RULE

CRITICAL EXAM PRINCIPLE: LEVEL BALANCING
All data flows entering or leaving a parent process on a higher-level DFD **MUST** match the net data flows entering or leaving the child diagram when that process is exploded. Unmatched flows across levels violate system balance.

8. LOGICAL VS. PHYSICAL DFDS

Dimension	Logical DFD	Physical DFD
Core Focus	WHAT the system does (Business domain rules).	HOW the system is built (Implementation details).
Processes	Business activities independent of implementation.	Software modules, manual steps, automated scripts.
Data Stores	Logical data collections (e.g., `Orders`).	Database tables, SQL DBs, flat files, paper files.
Controls	Omits routine operational controls.	Includes validation, security checks, temp files.

9. STEP-BY-STEP WORKED CASE STUDIES

Case Study 1: E-Commerce System (Daraz Platform)

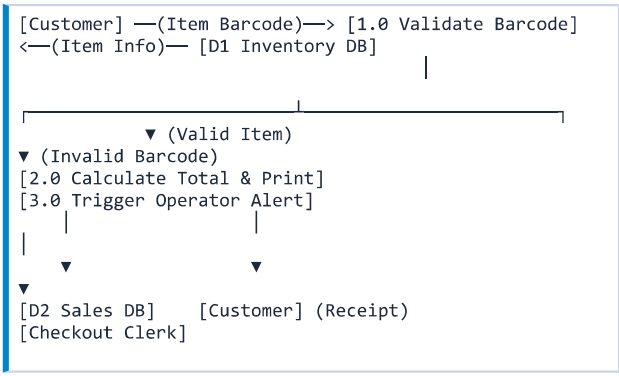
Context Diagram (Level 0):

- *Entities:* Customer, Vendor, Payment Gateway.
- *Flows:* User Credentials & Order Request → `0 Daraz Platform` → Order Confirmation & Payment Status.

Level 1 Decomposition:

- `1.0 Authenticate User` → Interacts with `D1 User Credentials`
- `2.0 Search & Browse Items` → Reads `D2 Product Catalog`
- `3.0 Process Order Cart` → Writes to `D3 Pending Orders`
- `4.0 Execute Payment` → Interfaces with `Payment Gateway` & writes `D4 Transactions`

Case Study 2: Supermarket POS Assignment Solution



10. DATA DICTIONARY & PROCESS SPECIFICATIONS

DFDs are complemented by a **Data Dictionary** and process specifications:

- **Data Dictionary Entry:** Defines data structures, e.g., `Customer_Address = House_No + Street + City + Postal_Code`.
- **Structured English:** Uses IF-THEN-ELSE to specify primitive process logic.
- **Decision Tables & Trees:** Tabular/graphical mapping of complex business rule combinations.

⚠️ COMMON EXAM PITFALLS & ILLEGAL PATTERNS

- **Black Hole Process:** A process node that has incoming data flows but NO outgoing data flows. (Data disappears!)
- **Miracle Process:** A process node that produces outgoing data flows without ANY incoming data flows. (Data created from nothing!)
- **Grey Hole Process:** A process where output data cannot be generated from the given input data (insufficient inputs).
- **Direct Entity-to-Store Link:** Connecting an External Entity straight to a Data Store without an intermediate Process. **ILLEGAL**
- **Data Store at Level 0:** Including a data store on the Context Diagram. Data stores must only appear at Level 1 or lower.
- **Verb-less Process Names:** Naming a process with a noun phrase (e.g., "Customer Record") instead of an active verb phrase (e.g., "Update Customer Record").

💡 EXAM REVISION SHORTCUTS & NAMING FORMULAS

- **Entity:** Noun (e.g., `Student`, `Library`)
- **Process:** Imperative Verb + Noun (e.g., `Calculate Grade`)
- **Data Store:** Noun Phrase + Prefix (e.g., `D1/Student File`)
- **Data Flow:** Specific Noun Phrase (e.g., `Course Request Form`)
- **Process Numbering Hierarchy:**
  - Level 1: ***1.0, 2.0, 3.0***
  - Level 2 (for 2.0): ***2.1, 2.2, 2.3***
  - Level 3 (for 2.1): ***2.1.1, 2.1.2***

11. 📝 PRACTICE SELF-ASSESSMENT QUIZ

Q1: Why is a direct arrow between two Data Stores illegal in DFD rules?

```
Answer: Data cannot move autonomously between data
stores. A Process is always required to read data from
one store, transform it, and write it to another store.
```

Q2: Identify the error in a process labeled "Customer Invoices".

```
Answer: "Customer Invoices" is a noun phrase
(appropriate for a Data Store). A Process label must
use an active Verb + Noun phrase, e.g., "Generate
Customer Invoices".
```

Q3: How does level balancing preserve architectural integrity?

```
Answer: Level balancing guarantees that boundary inputs
and outputs stay consistent when expanding a high-level
process into child diagrams, preventing lost or
orphaned data flows.
```

Q4: True or False? Data Stores can appear on a Level 0 Context Diagram.

```
Answer: FALSE. Level 0 shows only the overall system
process (Process 0), external entities, and major
boundary flows. Data stores are introduced at Level 1.
```

HIERARCHICAL SOFTWARE ARCHITECTURE

1. CORE CONCEPTS & OVERVIEW

In **Hierarchical Software Architecture**, complex software systems are decomposed into logical sub-systems/modules structured in vertical levels. Modules at higher levels invoke services or methods of modules at lower levels.

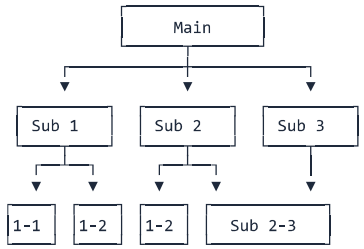
- **Procedural Paradigm:** Uses functions, subroutines, and procedures.
- **Object-Oriented Paradigm:** Uses classes, methods, and service objects.

2. KEY HIERARCHICAL STYLES

```
Hierarchical Architectures
  ├── 1. Main-Subroutine Architecture
  ├── 2. Master-Slave Architecture
  ├── 3. Layered Architecture (Strict vs. Relaxed)
  └── 4. Virtual Machine Architecture
```

3. STYLE I: MAIN-SUBROUTINE

Decomposes system functionality top-down into a hierarchy of subroutines. Main program acts as the root coordinator.



Data Passing Mechanisms

- **Pass by Value:** Subroutine receives copies of caller data. Safe against accidental modifications.
- **Pass by Reference:** Subroutine receives memory pointers. Allows subroutines to directly mutate state.

Real-World Example: Payroll System

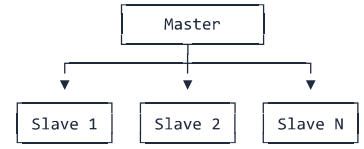
- `Main Process:` Executes monthly payroll pipeline.
- `Subroutines:` 1. Fetch Employee Data → 2. Calculate Deductions & Taxes → 3. Generate Payslip PDF.

Pros & Limitations

- **Benefits:** Intuitive top-down functional refinement; simple to understand for procedural workflows.
- **Limitations:** Globally shared data introduces security & state corruption risks; high degree of tight coupling makes change propagation difficult.

4. STYLE II: MASTER-SLAVE

A **Master** component divides work into independent subtasks, delegates them to multiple **Slave** components, and aggregates the returned sub-results into a final output.



Core Architectural Drivers & Uses

1. **Fault Tolerance & High Availability:** Slaves perform identical tasks in parallel. If one slave crashes or drops off, system remains operational without downtime (Replication).
2. **Computational Accuracy (High Assurance):**
   • Uses *non-identical slave implementations* (e.g., distinct algorithms written by independent teams).
   • Requires ≥ 3 slaves.
   • Master compares output: if results match, return to caller; if mismatch, invoke **majority voting** or throw an exception.
3. **Computationally Intensive Tasks:** Big data analysis or parallelizable tasks divided across slaves to maximize hardware core utilization.

Avionics Case Study: Triple Modular Redundancy (TMR)

In aircraft Flight Control Systems, 3 independent slave computers compute the flight path. The Master controller uses majority voting: if 2 slaves output coordinate A and 1 outputs B, result A is executed and the faulty slave is logged.

5. STYLE III: LAYERED ARCHITECTURE

Decomposes system into horizontal layers, each representing a distinct level of abstraction and holding sole responsibility for its domain.

- **Layer 1 (Bottom):** Interacts directly with hardware, OS, or DB.
- **Layer N (Top):** Interacts directly with external clients/UI.

Dynamic Execution Flow

A high-level request at Layer N translates into a cascading tree of lower-level requests to Layer N-1, down to Layer 1. Lower layers combine raw data and return refined abstractions upwards.

Strict vs. Relaxed Layering

Property	Strict Layering	Relaxed Layering
Allowed Calls	Layer *j* can *only* call Layer *j* – 1.	Layer *j* can call *any* lower layer (*j* – 1, *j* – 2, ...).
Coupling	Low; maximum isolation.	Higher coupling; bypasses intermediate layers.
Trade-off	Higher overhead; pass-through code.	Better performance & efficiency; harder to maintain.

Error Handling & Abstraction Leakage

Critical Rule: Low-level errors must *never leak* out to higher layers. A layer must catch raw lower-level errors (e.g., `SqlException`) and map/wrap them into domain-appropriate higher-level abstractions (e.g., `UserAuthenticationFailedException`).

6. CASE STUDY: ISO/OSI 7-LAYER MODEL

Layer #	Name	Primary Responsibility
7	**Application**	Network services for client applications.
6	**Presentation**	Data formatting, encryption, compression.
5	**Session**	Establishes, manages, terminates sessions.
4	**Transport**	End-to-end connection (TCP/UDP).
3	**Network**	Routing and IP addressing.
2	**Data Link**	Node-to-node framing & MAC addressing.
1	**Physical**	Raw binary bitstream transmission across media.

7. STYLE IV: VIRTUAL MACHINE (VM)

An abstraction layer that emulates target hardware/OS. Program code compiles to intermediate **bytecode**, which the VM executes on any underlying host system.

- **Known Examples:** Java Virtual Machine (JVM), .NET Common Language Runtime (CLR).
- **Primary Advantage:** Complete platform independence ("Write Once, Run Anywhere").

⚠️ **Common Exam Traps & Pitfalls:**

- **Master-Slave Voting Rule:** Fault Tolerance requires identical slave tasks; Accuracy/Diversity requires *different algorithms/implementations* across at least 3 slaves.
- **Error Leakage:** Allowing raw SQL/OS exception codes to bubble up to UI layers violates Layered Architecture abstraction rules.

💡 **Exam Mnemonic Shortcuts:**

• ISO/OSI Layers: **"All People Seem To Need Data Processing"** (Application, Presentation, Session, Transport, Network, Data Link, Physical).
• TMR = **Triple Modular Redundancy** (3 slaves + majority voting).

📝 SELF-ASSESSMENT QUICK CHECK

Q1: Why are non-identical slaves used in Master-Slave accuracy systems?

Ans: To prevent systematic algorithmic bugs. Diversity ensures software bugs in one implementation don't affect all slaves, allowing majority voting to succeed.

Q2: Contrast Strict vs. Relaxed Layering in performance terms.

Ans: Relaxed layering improves performance by letting higher layers bypass intermediate pass-through layers to invoke low-level utilities directly.

SERVICE-ORIENTED ARCHITECTURE (SOA)

1. CORE CONCEPTS & OVERVIEW

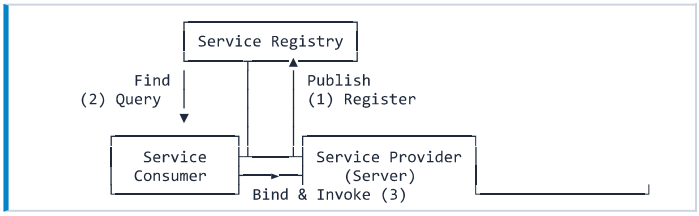
Service-Oriented Architecture (SOA) is an architectural style that structures software applications as a collection of **loosely coupled**, **reusable**, and **interoperable** services accessible over a network.

- **Service:** A self-contained unit of functionality executing a distinct business capability (e.g., payment, authentication).
- **Loose Coupling:** Services operate independently without direct internal dependencies on other implementations.
- **Network Access:** Exposed via standardized interfaces across network protocols.

2. THE 5 ESSENTIAL PARTS OF SOA

1. **Services:** Autonomous building blocks carrying out specific business logic or data processing tasks.
2. **Service Provider:** The entity/system that creates, hosts, and exposes services to external callers.
3. **Service Consumer:** Any application or component that requests and consumes exposed services.
4. **Service Registry:** A centralized directory storing service definitions, endpoints, and dynamic discovery metadata.
5. **Enterprise Service Bus (ESB):** Middleware backbone enabling centralized routing, transformation, and protocol translation.

3. THE DYNAMIC SOA TRIANGLE OPERATIONS



The Three Dynamic Operations

- **1. Publish:** Service Provider registers its endpoint metadata and interface contract (e.g., WSDL) to the Service Registry.
- **2. Find:** Service Consumer queries the Service Registry to locate an appropriate service matching specific functional requirements.
- **3. Bind & Invoke:** Consumer uses retrieved contract metadata to directly connect and interact with the Service Provider.

4. ENTERPRISE SERVICE BUS (ESB)

The **ESB** acts as an architectural middleware backbone that integrates heterogeneous enterprise systems using messaging, transformation, and protocol mediation.

Core ESB Function	Architectural Purpose
Message Routing	Directs messages from sender to target based on content or service rules.
Message Transformation	Converts payloads between formats (e.g., XML to JSON, schema mapping).
Protocol Mediation	Enables systems using disparate protocols (HTTP, JMS, FTP) to communicate.
Service Orchestration	Coordinates multiple underlying granular services into composite workflows.
Security & Auth	Enforces central authentication, authorization, and message encryption.
Error Handling	Manages retries, dead-letter queues, circuit breaking, and fault tolerance.
Monitoring & Logging	Tracks real-time system metrics, SLA adherence, and traffic performance.

5. COMMUNICATION PROTOCOLS IN SOA

SOA relies on standard network protocols to facilitate interoperability across heterogeneous environments:

- **SOAP:** XML-based messaging protocol providing high enterprise security and transaction support.
- **REST:** Lightweight HTTP architectural style manipulating resources using standard verbs (GET, POST, PUT, DELETE).
- **JMS:** Java Message Service for asynchronous, queue-based message decoupling.
- **HTTP / HTTPS:** Universal transport protocol carrying SOAP or REST payloads.
- **CORBA:** Legacy distributed object RPC protocol (superseded by SOAP and REST).

6. DEEP-DIVE: SOAP PROTOCOL

Simple Object Access Protocol (SOAP) is a strict, protocol-independent messaging protocol using XML payloads.

Key Characteristics

- **Protocol-Independent:** Operates over HTTP, HTTPS, SMTP, FTP, or JMS.
- **XML-Based & Interoperable:** Language and platform neutral.
- **Extensible Standard:** Native support for WS-Security, WS-ReliableMessaging, and WS-AtomicTransaction.

SOAP Message Structural Anatomy

```
<?xml version="1.0" encoding="UTF-8"?>
<soapenv:Envelope xmlns:soapenv="..." xmlns:web="...">
  <!-- 1. Header (Optional): Metadata, Auth, Routing -->
  <soapenv:Header>
    <web:AuthHeader>
      <web:Username>user123</web:Username>
      <web:Password>pass123</web:Password>
    </web:AuthHeader>
  </soapenv:Header>
  <!-- 2. Body (Mandatory): Core Request Payload -->
  <soapenv:Body>
    <web:GetWeather>
      <web:City>New York</web:City>
    </web:GetWeather>
  </soapenv:Body>
  <!-- 3. Fault (Optional): Error Diagnostics -->
  <soapenv:Fault>
    <faultcode>soapenv:Client</faultcode>
    <faultstring>Invalid city name</faultstring>
  </soapenv:Fault>
</soapenv:Envelope>
```

7. SERVICE DIVISION & DESIGN PRINCIPLES

Design Criterion	Implementation Strategy
Business Capabilities	Group services around domain functional boundaries (e.g., Billing, Inventory).
Granularity	Balance simple single-purpose fine-grained tasks with composite operations.
Reusability	Expose capabilities designed for generic use across multiple client applications.
Separation of Concerns	Ensure each service handles a single distinct area of system logic.

8. EVOLUTION: SOA VS. MICROSERVICES

Dimension	Traditional SOA	Microservices
Architecture	Enterprise-wide sharing; Smart Pipes (ESB).	Application-level; Dumb Pipes / Smart Endpoints.
Data Storage	Shared Enterprise Database.	Database per Service (Isolated).
Protocol	Heavyweight (SOAP, WSDL, XML).	Lightweight (RESTful JSON, gRPC).
Coupling	Loosely coupled via central bus.	Fully autonomous independent services.

⚠️ **Common Exam Pitfalls & Misconceptions:**

- **Transport Trap:** SOAP is NOT restricted to HTTP; it is transport-agnostic (works over JMS, SMTP, TCP).
- **ESB Bottleneck:** Over-centralizing business logic inside the ESB creates single points of failure and deployment bottlenecks.
- **SOAP Fault Element:** Occurs inside the `Envelope` under `Body` or alongside it to transmit structured errors.

💡 **High-Yield Exam Memory Shortcuts:**

• SOA Dynamic Triangle: **Publish** → **Find** → **Bind**.
• SOAP Anatomy: **Envelope** wrappers contain **Header** (Meta/Auth), **Body** (Payload), and **Fault** (Errors).
• ESB Core Mission: **Transform, Route, Mediate** heterogeneous applications.

📝 SELF-ASSESSMENT QUICK CHECK

Q1: Why is an ESB described as enabling "Smart Pipes"?

Ans: Because the ESB embeds routing, protocol transformation, and orchestration logic inside the central middleware network layer.

Q2: What element in a SOAP message conveys authentication credentials?

Ans: The optional `<soapenv:Header>` element.

Q3: How does Service Registry enable dynamic service binding?

Ans: Consumers query the registry at runtime to find endpoint locations and WSDL interface specs dynamically without hardcoding URLs.

MICROSERVICES ARCHITECTURE

1. MOTIVATION & ARCHITECTURE EVOLUTION

Motivation: Large applications become difficult to manage over time as new features accumulate, large developer teams overlap on single codebases, modules require differing scaling levels, and minor changes necessitate risky full-system re-deployments.

```
[Monolith Risk Pipeline]
Bug in Payment -> Test Full App -> Deploy Full App -> High Risk
```

Architectural Evolution Stages

Systems evolve through key structural paradigms:

```
Single Program -> Layered Architecture -> Monolithic App -> Service-
Oriented Architecture (SOA) -> Microservices -> Cloud-Native Systems
```

Core Definition

A **Microservice** is a small, independently deployable service that implements *one single business capability* and communicates with other services using well-defined APIs.

2. COMPARATIVE ARCHITECTURAL PARADIGMS

Monolithic vs. Microservices Architecture

Aspect	Monolithic	Microservices
Structure	One unified codebase	Many small autonomous services
Deployment	Single deployment unit	Independently deployed services
Scaling	Scale entire application	Fine-grained per-service scaling
Database	Shared central database	Database-per-service pattern
Tech Stack	Single, uniform stack	Polyglot (different stacks)
Failure Isolation	Single module can crash app	Faults contained locally
Complexity	Simpler initial setup	Higher operational complexity

Modular Monolith vs. Microservices

Aspect	Modular Monolith	Microservices
Organization	Modules in 1 codebase	Separate independent services
Deployment	Single deployment unit	Multiple deployment units
Call Style	In-process method calls	Network calls / Messages
Data Ownership	1 DB (logical isolation)	Database-per-Service
Op. Cost	Lower overhead	Higher infrastructure cost
Best fit	Medium / Early-stage	Large, multi-team, high scale

3. KEY CHARACTERISTICS & API GATEWAY

- **Single Responsibility:** One focused purpose per service.
- **Loose Coupling:** Minimal knowledge/dependency on internal details of other services.
- **High Cohesion:** Related business domain functions grouped together.
- **Autonomy:** Services develop, test, and evolve independently.
- **Decentralized Data:** Each service owns and controls its database.

API Gateway Pattern

Serves as the single entry point for all external client requests (Web, Mobile):

- **Request Routing & Aggregation:** Combines results from multiple microservices into a single response.
- **Cross-Cutting Concerns:** Centralized Authentication/Authorization, Rate Limiting, Logging, Caching, and TLS Termination.

4. SERVICE COMMUNICATION PROTOCOLS

Synchronous vs. Asynchronous Communication

- **Synchronous (REST / gRPC):** Caller blocks and waits for a response. Simple to implement, but creates tight temporal coupling and potential network delays ($Latency_{total} = \sum t_i$).
- **Asynchronous (Event-Driven / Messaging):** Sender publishes events to a Message Broker (Kafka/RabbitMQ) without waiting. Promotes low temporal coupling, but requires tracking eventual consistency.

Communication Protocols Comparison

Aspect	REST	gRPC	Messaging
Style	Req / Res	RPC	Event / Message
Data Format	JSON / XML	Protocol Buffers	JSON, Avro, Protobuf
Performance	Good	Very Fast (Binary)	Depends on broker
Best For	Public APIs, Web	Internal high-perf	Workflows, Events

Service Discovery Pattern

Because service instances start, stop, scale, and move dynamically (changing IP addresses), a **Service Registry** (e.g., Netflix Eureka, Consul, K8s DNS) registers instance network locations dynamically so callers can discover endpoints.

5. DATA MANAGEMENT & THE SAGA PATTERN

Database-per-Service: Each service owns its database exclusively. Direct cross-database joins are prohibited; data access must occur strictly via published APIs or events.

Distributed Transactions & The Saga Pattern

Traditional ACID transactions (2-Phase Commit) do not scale across network boundaries. The **Saga Pattern** manages multi-service workflows via a sequence of local transactions. If a step fails, **Compensating Actions** are executed in reverse to undo previous state changes.

```
[Saga Execution Flow]
1. Create Order (Order Svc) -> Success
2. Process Payment (Payment Svc) -> Success
3. Update Stock (Inventory Svc) -> FAILED ❌
4. Compensating Action: Issue Refund & Cancel Order! 🔄
```

6. CONTAINERS, KUBERNETES & OBSERVABILITY

Containers & Docker

Containers package application code, dependencies, and configuration into a lightweight, portable image, guaranteeing identical runtime behavior across environments.

Kubernetes (K8s) Orchestration

Kubernetes automates multi-container management, providing:

- **Auto-Scaling & Load Balancing:** Distributes traffic across Pods.
- **Self-Healing:** Restarts failed containers automatically.
- **Rolling Updates & Rollbacks:** Zero-downtime releases.

Observability Triad

- **Centralized Logs:** Aggregated log collection (ELK/EFK stack).
- **Metrics:** Numerical system performance telemetry (Prometheus/Grafana).
- **Distributed Tracing:** Tracks request propagation across network boundaries using a correlation ID (Jaeger/Zipkin).

7. DECISION FRAMEWORK & BEST PRACTICES

When to Use: Large applications with dedicated multi-team organization, requirement for independent scaling, high continuous delivery demands, and mature DevOps capabilities.

When to Avoid: Small applications, small teams, rapidly shifting requirements, or limited DevOps expertise. *Rule of thumb: Start with a Modular Monolith!*

⚠️ **COMMON STUDENT PITFALLS & EXAM TRAPS**

- **Direct DB Access:** Allowing Service A to query Service B's database directly breaks encapsulation and autonomy. Always use APIs/events!
- **Distributed Monolith:** Splitting code into microservices while keeping synchronous REST call chains creates cascading network latency without benefits.
- **2PC Fallacy:** Assuming standard 2-Phase Commit transactions work in microservices. Use the Saga Pattern with Compensating Actions!

💡 **HIGH-YIELD EXAM STRATEGY SHORTS**

- **Core Trade-off Formula:** Microservices solve *Scaling & Operational Autonomy* problems, but introduce *Distributed System Complexity* (Network latency, partial failures, data consistency).
- **Security Layering:** Authenticate client at API Gateway (OAuth2/JWT); use mTLS and token propagation for inter-service security.

8. STEP-BY-STEP ARCHITECTURAL SCENARIO

🛠️ **WORKED DESIGN EXAMPLE: University Student System**

Requirement: Design a scalable University System (Admissions, Courses, Payments, Exams, Notifications).

Solution Architecture Blueprint:

1. **API Gateway:** Single entry point for Student Portal & Mobile App. Handles Auth & Rate Limiting.
2. **Service Decomposition:**
   - *Student Service:* Profiles (Relational DB)
   - *Course Reg Service:* Enrollment (Relational DB)
   - *Payment Service:* Tuition fees (ACID DB)
   - *Notification Service:* Email/SMS alerts (NoSQL)
3. **Communication Strategy:**
   - REST/gRPC for synchronous course lookup.
   - Message Broker (Kafka) for async events: `CourseEnrolled` event triggers `PaymentRequested` and `NotificationSent`.

9. 📝 SELF-ASSESSMENT QUICK CHECK

1. **Define Microservices Architecture.**
   Ans: An architectural style structuring an app as a collection of small, autonomous, independently deployable services organized around business capabilities communicating via APIs.
2. **Why is Database-per-Service mandatory?**
   Ans: It ensures loose coupling and service autonomy so schema changes in one service do not break or block other services.
3. **Explain how the Saga pattern resolves failure in async payment processing.**
   Ans: By executing a sequence of local transactions; if payment fails, compensating actions (e.g., cancel reservation, issue refund) roll back prior states.